

2025年度
修士論文

SN開発におけるデバッグシステムの開発

指導教員	アサノ デービッド	教授
	不破 泰	肩書き

2025年1月30日提出

信州大学大学院 総合理工学研究科 工学専攻 情報数理・融合システム分野
24W6047A
辻村篤志

あらまし

近年、無線センサネットワーク（WSN）や IoT は、環境モニタリングやスマートシティなど多様な分野での活用が進み、実機上で通信プロトコルを検証する重要性が高まっている。先行研究では、UML 状態遷移図から通信プロトコルのコードを自動生成し、無線通信デバイス上で動作検証できる環境が構築されていた。しかし、デバッグ方法はエディタのコンソールへのログ出力に依存しており、通信処理が高速かつ複雑に進行する場合に状態遷移や内部変数の推移を逐次追跡することが困難であった。

本研究では、実機で実行されるプロトコルの内部動作を直感的に把握できるデバッグ支援システムを提案し、ブラウザ上で動作する可視化デバッグ環境として実装した。提案システムは、状態遷移ログと変数更新ログを統一形式で出力し、ログ解析により状態列と各ステップに対応する変数値スナップショット列を再構成する。これにより、状態遷移図のハイライト表示、変数ウォッチ表示、ステップ実行による逐次確認を、同一のステップ番号に基づいて連動させる。加えて、設計に用いる状態遷移図をデバッグシステム上に表現することで、通信状態の遷移をより視覚的にとらえることを可能にした。また、同期モードでは、2 台のデバイスから同時に受信したログに時刻情報を付与し、イベント列を統合して同期タイムラインを生成することで、同一時刻における 2 台のノードの状態と変数を比較可能とした。

評価では、実行ログを用いて単一ノードの追跡性向上と、同期タイムラインによる複数ノードの相互作用把握を確認し、提案システムが実機検証におけるデバッグ作業の効率化に有効であることを示した。

目次

第1章	はじめに	1
1.1	研究背景	1
1.2	当研究室における先行研究	1
1.3	先行研究の課題	2
1.4	本研究の目的	2
第2章	先行研究の提案システム	3
2.1	システムの概要	3
2.2	実現事項	4
2.3	課題	4
第3章	提案システムの概要と設計方針	5
3.1	設計方針	5
3.2	システムの構成	6
3.3	各コンポーネントの概要	7
3.3.1	コード生成コンポーネント	8
3.3.2	組込みデバイスコンポーネント	8
3.3.3	デバッグ支援システムコンポーネント	9
第4章	デバッグ支援システムの設計・実装	10
4.1	デバッグ支援システムの概要	10
4.2	実装技術と採用理由	10
4.2.1	フロントエンドフレームワーク	10
4.2.2	UI フレームワーク	11
4.2.3	状態遷移図の描画ライブラリ	11

4.2.4	ブラウザ上でのシリアル受信	11
4.3	動作モードとシステム構成	11
4.3.1	動作モードの分類	11
4.3.2	モード間で共通に用いる設計要素	12
4.4	ファイル解析モードの設計・実装	13
4.4.1	ファイル解析モードの概要	13
4.4.2	ファイル解析モードの UI 設計	13
4.5	シリアル受信モードの設計・実装	14
4.5.1	シリアル受信モードの概要	14
4.5.2	シリアル受信モードの UI 設計	15
4.5.3	ブラウザ上でのシリアルモニタ実装	17
4.5.4	同期タイムラインの生成	17
4.6	ログ設計	18
4.6.1	ログ生成の設計	18
4.7	ログ解析の設計（共通処理）	20
4.7.1	概要（解析の目的と処理の流れ）	20
4.7.2	入力ログ形式とログ行の分類	21
4.7.3	解析結果として構築するデータ構造	22
4.7.4	変数値スナップショットの再構成方針（継承と更新）	22
4.7.5	行処理アルゴリズム（擬似コード）	23
4.7.6	解析結果例	24
4.8	ステップ番号に基づく共通表示ロジック	24
4.8.1	共通の前提：ステップ番号 i と状態スナップショット	24
4.8.2	ステップ実行のロジック	25
4.8.3	変数ウォッチ機能のロジック	26
4.8.4	状態遷移表示機能（ハイライト表示）	26
4.8.5	Mermaid による状態遷移図描画（構文生成ロジック）	27
第 5 章	評価	29
5.1	評価の目的と方針	29
5.1.1	評価の目的（何を確かめたいか）	29

5.1.2	評価対象	29
5.1.3	評価で用いるデータ（実験対象）	30
5.2	単一ノードデバッグの評価（ログモード／単一ノード受信の共通部分）	30
5.2.1	評価シナリオ	30
5.2.2	ログ入力と解析結果の生成（states/vars）	31
5.2.3	状態遷移ログ表示（ログ入力の可視化）	34
5.2.4	変数ウォッチ表示（状態ステップに対応した変数値追跡）	34
5.2.5	状態遷移図表示（JSON 入力→Mermaid 変換→描画）	36
5.2.6	ステップ操作・スライダ操作（表示要素の同期更新）	38
5.3	同期モード固有機能の評価（2 ノード同期タイムライン）	39
5.3.1	評価シナリオ	39
5.3.2	シリアルモニタによるログ受信	40
5.3.3	同期タイムライン生成と表示	40
5.3.4	複数ノード比較の有効性（定性的観察）	42
5.4	定性的評価	43
5.4.1	単一ノードデバッグに対する効果	43
5.4.2	同期デバッグに対する効果	44
5.4.3	まとめ	44
第6章	考察	45
6.1	有効性の要因	45
6.1.1	状態遷移単位への再構成	45
6.1.2	変数値スナップショットの継承と更新	45
6.1.3	同期モードにおける統合表示	45
6.2	システムの限界・制約	46
6.2.1	ログ設計への依存	46
6.2.2	同期タイムラインに関する制約	46
6.3	今後の課題	46
6.3.1	ログ形式・解析対象の拡張	46
6.3.2	同期精度および比較支援の拡張	46
6.3.3	状態遷移図表現への対応	46

目次	vi
第7章 まとめ	47
7.1 本研究のまとめ	47
7.2 得られた知見と今後の展望	47
参考文献	48

図 目 次

2.1	先行研究システムの概要図	3
3.1	従来のデバッグ方法（コンソール出力）	6
3.2	提案システムの概要図	7
3.3	どこでモデム（左）と GPS モジュール（右）	9
4.1	モード選択のフローチャート	12
4.2	ファイル解析モードのレイアウト	14
4.3	シリアルモニタのレイアウト	15
4.4	シリアルポート選択ドロップダウン	16
4.5	同期モードのレイアウト	17
4.6	同期モードのタイムライン（例 1）	18
4.7	同期モードのタイムライン（例 2）	18
4.8	ログ解析の処理フロー	21
4.9	変数値スナップショットの継承と更新	23
5.1	状態遷移ログ表示（左：10 行表示、右：30 行表示）	34
5.2	変数ウォッチ表示（変数選択）	35
5.3	変数ウォッチ表示（ステップによる変数の変化）	35
5.4	Mermaid による状態遷移図の描画例	38
5.5	ステップ実行の様子	39
5.6	シリアルモニタによるログ受信の様子（左：中継器、右：端末）	40
5.7	同期化された状態遷移表示例	42
5.8	同期デバッグ実行例（タイムライン選択に連動した複数ノードの同時表示）	43

表 目 次

第 1 章

はじめに

1.1 研究背景

近年、無線センサネットワーク（Wireless Sensor Network：WSN）や IoT（Internet of Things）は、環境モニタリングやスマートシティ、インフラ監視、農業、ヘルスケアなど、さまざまな分野での応用が期待されている。実際、IoT 全体の接続デバイス数は急速に増加している。IoT Analytics によれば、2023 年時点で約 166 億台、2024 年には約 188 億台に達し、2030 年には約 400 億台に到達すると予測されている [1]。また、WSN の普及も市場規模の面から拡大を続けており、Grand View Research の報告では、産業用 WSN（Industrial WSN）の市場は 2023 年に約 51.9 億ドルと推定され、2024 年から 2030 年にかけて年平均 12.1%の成長が見込まれている [2]。

しかし、これらのシステムを構築するためには、複雑な通信プロトコルの設計やデータ処理の設計および実装、動作検証が求められ、開発者には幅広い専門知識と多大な工数が必要となる。さらに、無線通信環境の構築やマイコン設定など導入時のハードルも高いため、これらの研究や開発はシミュレーション環境やエミュレータ上での検証にとどまることが多い。しかし、シミュレーション環境では実機特有の挙動や外部環境の影響を完全に再現することが難しく、実機での検証が不可欠である場合も多いため、実機でのプロトコル検証を容易にするための支援環境の整備が求められている。

1.2 当研究室における先行研究

当研究室の先行研究では、UML の一種である**状態遷移図**上で無線通信プロトコルの動

作を定義し、そこから自動生成したプログラムを汎用ハード上で動作させ、プロトコルを実機で動作検証できる環境が構築されていた（旭ら [3]、小林ら [4]）。

1.3 先行研究の課題

このシステムにより簡易的に通信プロトコルの動作検証が可能となったが、デバッグ方法はエディタ上のコンソールへのテキストログ出力に依存しており、通信処理が高速かつ複雑に進むため逐次的な状態遷移を追跡することが難しかった。その結果、複雑な動作を含むプロトコルに対しては、異常動作の原因究明や、通信動作を把握しづらく、開発効率や教育的利用の観点では十分でない点が課題として残されていた。

1.4 本研究の目的

本研究の目的は、先行研究で課題となっていたデバッグ効率の低さを改善し、実機でのプロトコル検証をより効率的かつ視覚的に行える環境を実現することである。そのために、無線通信デバイス上で実行された動作をログとして出力し、それを可視化・ステップ実行できる仕組みを開発した。これにより、プロトコルの動作を逐次的に把握でき、異常動作の原因究明や通信動作の把握を効率化するとともに、教育やチーム開発、応用実証に適した支援環境を提供することを目指す。

第 2 章

先行研究の提案システム

2.1 システムの概要

先行研究で提案されたシステムの概要図を図 2.1 に示す。先行研究では、無線通信プロトコルの設計・実装・動作検証を支援するシステムが提案されてきた。このシステムは主に次の二つのコンポーネントから構成されている。第一に、UML の状態遷移図を用いてプロトコルの動作を定義し、そこからファームウェア（実機上で動作するプログラム）を自動生成するコンポーネントである。第二に、生成されたファームウェアを実行するための環境（ハードウェア）を提供するコンポーネントである。これにより、状態遷移図で定義されたプロトコルが実機上で動作し、その動作を検証できる環境が提供されてきた。つまりユーザは状態遷移図さえ記述すれば、無線通信プロトコルの実装および動作検証を容易に行うことができる。

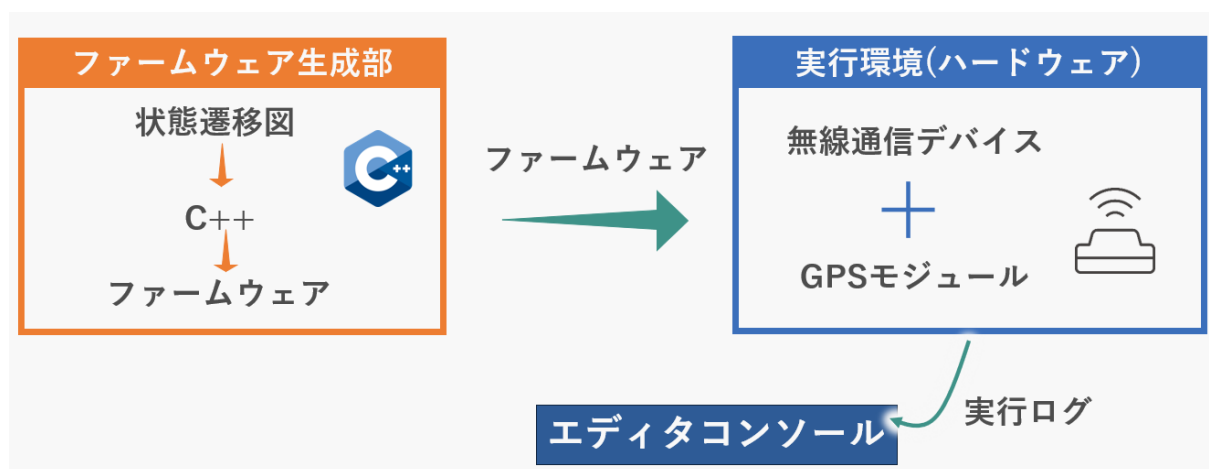


図 2.1: 先行研究システムの概要図

2.2 実現事項

先行研究システムでは、主に以下の機能が実現されている。

- 状態遷移図からのコード自動生成：UML の状態遷移図を用いてプロトコルの動作を定義し、そこから C++ プログラムを自動生成する機能が提供されている。これにより、プロトコル設計者は視覚的に動作を定義でき、手動でのコーディング作業を削減できる。
- 実機上での動作検証環境：生成されたコードは、無線モデム搭載のマイコンを組み合わせたハードウェア上で実行される。これにより、シミュレーション環境では捉えきれない実機特有の挙動を検証できる。

2.3 課題

先行研究システムには以下の課題が存在する。

- 状態遷移図の記法の制約：先行研究における状態遷移図からのコード自動生成は、特定の記法に依存しており、複雑なプロトコル動作を表現する際に制約がある。これにより、設計者が意図する動作を完全に反映できない場合がある。
- デバッグ効率の低さ：デバッグ方法がコンソールへのテキストログ出力に依存しており、通信処理が高速・複雑に進むため逐次的な内部状態（状態、変数値など）を追跡することが難しい。これにより、複雑な動作を含むプロトコルに対しては、異常動作の原因を把握しづらい。

これらの課題を解決するために、本研究ではデバッグ効率の向上に焦点を当て、実機でのプロトコル検証をより効率的に行える環境を提案する。なお、状態遷移図からのコード生成部については、[5] で改善が図られているため、そちらを参照されたい。

第 3 章

提案システムの概要と設計方針

本研究では、先行研究システムにおいて課題となっていたデバッグ効率の低さを改善することを目的として、新たなデバッグ支援システムを提案する。本章では、提案システムの概要と設計方針について述べる。まず、先行研究における課題を整理し、次に設計方針と満たすべき要求を示す。最後に、システム全体の構成と各コンポーネントの役割を説明する。

3.1 設計方針

先行研究システムでは、状態遷移図を用いて通信プロトコルを設計し、実機上で動作検証を行うことが可能であった。一方で、デバッグ方法は主に図 3.1 のようにコンソールへのログ出力に依存しており、状態遷移の流れや内部変数の変化を逐次的に把握することが難しいという課題があった。通信処理が高速に進行する場合、ログが短時間に大量出力されるため、必要な状態や変数の変化点を追跡しにくく、原因箇所の特定に手作業の負担が生じる。

本研究ではこの課題を踏まえ、実機上で実行される通信プロトコルの内部動作を、**状態遷移**に基づいて段階的に確認できるデバッグ支援環境の構築を設計方針とした。具体的には、プロトコルの実行状況をログとして取得し、それを解析して可視化することで、動作の流れをステップ単位で追跡できる仕組みを目指す。

設計上の要求を以下にまとめる。

- プロトコルは状態遷移に基づいて動作すること。

```
STATE:IDLE
current_slot = 0
STATE:RECEIVE_SLOT_INFO
STATE:CHECK_RECEIVED_PACKET
STATE:WAIT_OWN_SLOT
current_slot = 1
STATE:WAIT_OWN_SLOT
STATE:WAIT_OWN_SLOT
current_slot = 2
STATE:WAIT_RANDOM_DELAY
STATE:CREATE_PACKET
STATE:TRANSMIT
.
.
.
```

図 3.1: 従来のデバッグ方法（コンソール出力）

- ログは状態遷移ログと変数更新ログを解析対象とし、解析可能な統一された形式で出力されること。
- 状態遷移をステップ単位で追跡できること。
- 状態遷移図、ログ、変数値が同一ステップに基づいて同期して提示されること。
- 先行研究の枠組みと親和性を保ち、研究室内で継続利用および拡張が容易であること。
- 単一ノードの解析に加えて、複数ノードの比較にも拡張可能であること。

3.2 システムの構成

システム全体の構成を図 3.2 に示す。提案システムは、先行研究システムに対してデバッグ支援機能を付加する形で構成されている。全体としては、ファームウェアを生成す

るファームウェア生成部、通信プロトコルを実行する実行環境（組み込みデバイス）と、その動作を外部から確認するためのデバッグ支援システムからなる。

組み込みデバイスでは、状態遷移に基づいて通信プロトコルが実行され、その実行過程に関する情報がログとしてシリアル経由で出力される。デバッグ支援システム側では、出力されたログを収集して解析し、状態遷移や変数値の推移を可視化する。これにより、従来は把握が困難であった動作過程を、ステップ単位で確認可能とする。また、従来はPlatformIO上のシリアルモニタでログを確認していたが、本システムではブラウザ上で動作するシリアルモニタをデバッグ支援システムに実装し、ログの受信と解析を一元的に行えるようにした。

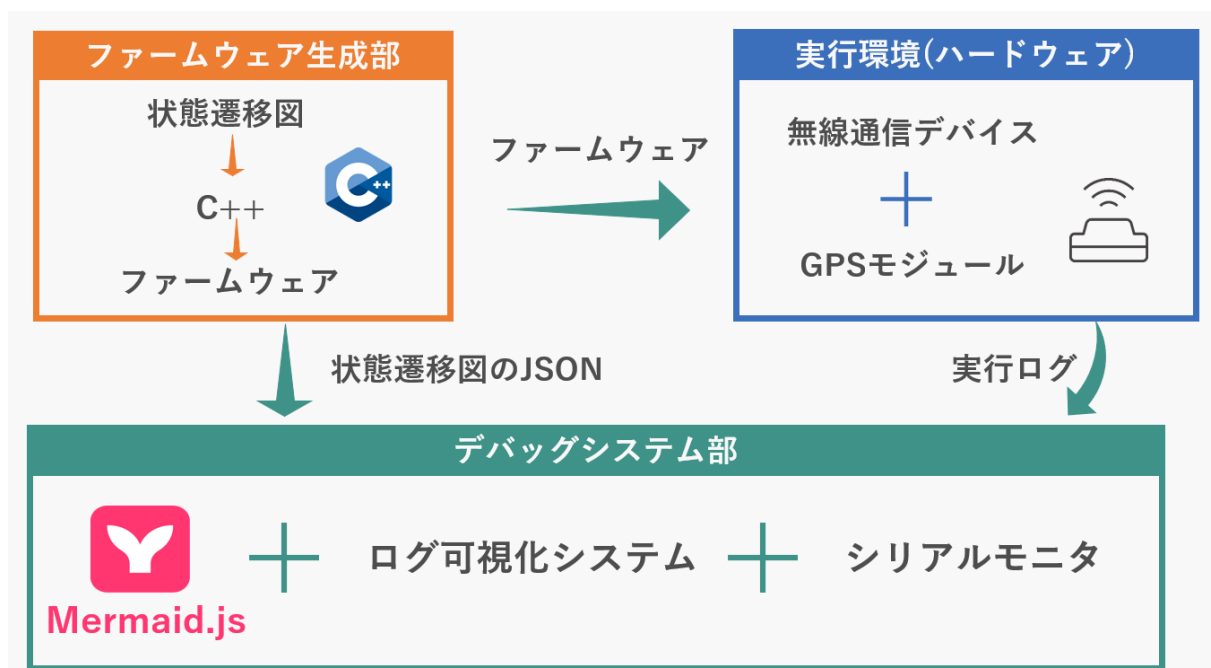


図 3.2: 提案システムの概要図

3.3 各コンポーネントの概要

提案システムは、主に以下のコンポーネントから構成されている。

- 状態遷移図からコードを自動生成するコンポーネント。
- 通信プロトコルを実行する組み込みデバイスコンポーネント。
- ログ収集と可視化を担うデバッグ支援システムコンポーネント。

なお、コード生成およびデバッグコンポーネントはPC上で動作し、組み込みデバイスは外部デバイスであることを留意されたい。

3.3.1 コード生成コンポーネント

コード生成コンポーネントは、UMLの状態遷移図を解析し、通信プロトコルの動作を表現するプログラムおよびファームウェアを自動生成する役割を担う。

本コンポーネントでは状態遷移図の構造を解析し、状態、遷移、イベントおよびアクションを抽出する。抽出した情報に基づき、通信プロトコルの動作を表現するC++コードを生成し、状態遷移はswitch-case文を用いて実装される。生成されたコードはPlatformIOを用いてビルドされ、ファームウェアが生成される。このファームウェアは次節で述べる組み込みデバイスに書き込まれた後、その上で実行される。

なお、本研究の主眼はコード生成手法そのものではなく、生成されたプログラムの実行ログに対するデバッグ支援環境の構築にあるため、コード生成手法の詳細については文献[5]を参照されたい。

3.3.2 組み込みデバイスコンポーネント

本システムにおけるプロトコルの実行環境である組み込みデバイスを図3.3に示す。本研究では、SAMD21マイコンと無線通信モジュールを一体化した無線通信デバイス（株式会社サーキットデザイン製どこでモデム）を用いる。

本デバイスは429MHz帯で動作する汎用的な無線通信モジュールであり、技術基準適合証明（技適）を取得しているため、実際に電波を用いた通信プロトコルの検証が可能である。また、429MHz帯は中山間地域において回折性能に優れており、低消費電力で長距離通信が可能なLPWA（Low Power Wide Area）通信に適している。この特性により、登山者見守りシステムなどへの応用も期待できる。さらに、本デバイスはマイコンと通信モジュールが一体化しているため、外部配線や追加ハードウェアを必要とせず、開発や実験環境の構築を容易に行える。また、本デバイスはPCとUSBを用いたシリアル通信が可能であり、デバッグ支援システムと連携してログを送信できる。

必要に応じてGPSモジュールを併用することで、位置情報の取得や1PPS信号を用いた時刻同期により、高精度同期を前提とするTDMA方式などにも対応可能となる。な



図 3.3: どこでモデム（左）と GPS モジュール（右）

お、GPS モジュールは使用しない場合には取り外すこともできる。また、その他のセンサを用いた組み込みシステムを構築する場合には、別途センサモジュールを接続して利用することも可能である。

3.3.3 デバッグ支援システムコンポーネント

デバッグ支援システムは、本研究の中核であり、組込みデバイスから出力される実行ログを収集し、解析結果を可視化して提示する役割を担う。従来はコンソール上に流れるログを人手で追跡する必要があり、状態遷移の流れや変数値の変化点を把握しにくいという課題があった。本システムでは、状態遷移ログと変数更新ログを解析対象として統一形式で扱い、ログを状態遷移の出現順に基づくステップ単位へ再構成することで、状態遷移図、状態ログ、変数値表示を同一ステップに基づいて同期させる。

さらに、ログの入力経路として、実行ログファイルアップロードに加えてブラウザ上で動作するシリアルモニタを備える。これにより、組込みデバイスの実行ログを受信し、そのまま同一画面内で解析と可視化へ移行できるため、ログ受信とデバッグ作業を一元化できる。また、複数ポートからのログを同時に取り扱うことで、複数ノード間の挙動比較に発展させることも可能である。

以上により、本コンポーネントは、実機上で進行する通信プロトコルの動作を段階的に確認し、原因箇所の特定や相互作用の理解を支援する。本コンポーネントの具体的な設計と実装については、次章で詳述する。

第 4 章

デバッグ支援システムの設計・実装

4.1 デバッグ支援システムの概要

本研究で提案するデバッグ支援システムは、ブラウザ上で動作するフロントエンドアプリケーションとして実装されている。本システムは、組み込みデバイスから得られる実行ログと状態遷移図情報を入力として、(1) 状態遷移の進行状況の可視化、(2) 変数値の追跡、(3) ステップ実行による逐次確認、(4) 複数ノード間の同期デバッグ、を支援することを目的とする。

本章では、まず実装技術と採用理由を述べ、次に動作モードを「ファイル解析モード」と「シリアル受信モード」に整理する。その後、モード間で共通に用いる設計要素として、ログ設計、ログ解析、ステップ番号に基づく共通表示ロジック、変数ウォッチ、状態遷移図描画とハイライト表示を順に説明する。

4.2 実装技術と採用理由

4.2.1 フロントエンドフレームワーク

UI およびロジックの構築には React.js（以下、React）を採用した。React はコンポーネント指向のフレームワークであり、画面要素を状態（state）と結び付けて管理できる。本研究のデバッグ支援システムでは、ステップ操作に伴い、状態遷移図のハイライト表示や変数値の表示が頻繁に更新される。このため、状態変化に応じた UI 更新を宣言的に

記述できる React は適している。さらに、表示部品をコンポーネントとして分割できるため、可読性・保守性の向上にも寄与する。

4.2.2 UI フレームワーク

UI のスタイリングおよびレイアウト設計には Bootstrap を使用した。Bootstrap は、あらかじめ定義された CSS クラスおよび UI コンポーネントを提供し、レスポンシブな画面設計を容易に実現できる。本システムでは、多数の情報（状態遷移図、ログ、変数値など）を同一画面上に整理して表示する必要があるため、グリッドシステムやカードコンポーネントを活用し、視認性と操作性の両立を図った。

4.2.3 状態遷移図の描画ライブラリ

状態遷移図の描画には Mermaid.js を用いる。Mermaid は図をテキスト（構文）として記述し、その文字列から図を生成できるため、状態遷移図 JSON から構文を生成する設計とすることで、自動描画とステップ連動のハイライト表示を実現できる。

4.2.4 ブラウザ上でのシリアル受信

シリアル受信モードでは Web Serial API を用い、ブラウザから直接シリアルポートに接続してログをリアルタイムに受信する。従来使用していた別アプリケーション（platformIO 等）のシリアルモニタを使用せず、ログの受信、解析、可視化を同一 UI 内で一元化できる点を重視した。

4.3 動作モードとシステム構成

4.3.1 動作モードの分類

本システムは入力経路に基づき、ファイル解析モードとシリアル受信モードの2つの動作モードを備える。図 4.1 に使用するモードの選択フローチャートを示す。

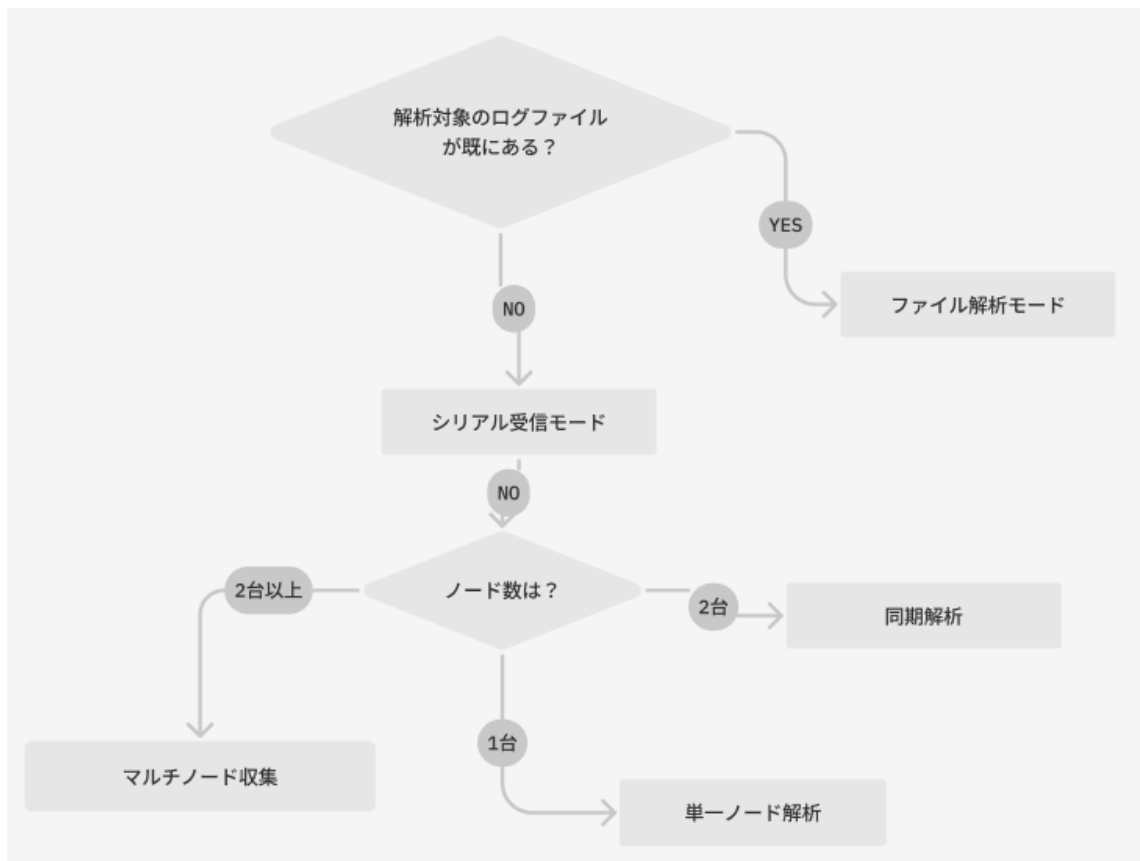


図 4.1: モード選択のフローチャート

ファイル解析モードは、既を取得済みのログファイルを入力として解析と可視化を行う。シリアル受信モードは、組込みデバイスからログを受信し、目的に応じて解析と可視化、または収集を行う。

シリアル受信モードはノード数と目的に応じて、単一ノード解析、同期解析（2台）、マルチノード収集（2台以上）の3形態に分かれる。単一ノード解析と同期解析は受信ログを解析して可視化デバッグを行う。マルチノード収集は、主としてログを受信してコピーを行うことを目的とし、解析は対象外とする。なお以上の3形態はユーザーが指定するものではなく、接続されているノード数と目的に応じて自動的に切り替わる。

4.3.2 モード間で共通に用いる設計要素

両モードはログの入力経路が異なるが、解析と可視化の中核処理は共通（ログ解析は一部異なる）である。共通に用いる要素を以下に示す。

- ログの形式：状態遷移ログと変数更新ログ（4.6 節）

- ログ解析：状態列と変数値スナップショット列の構築（4.7 節）
- 変数追跡：ウォッチ機能による変数値表示（4.8.3 節）
- 状態遷移可視化：状態遷移図の描画とハイライト表示（4.8.5 節、4.8.4 節）
- 共通表示ロジック：ステップ番号に基づく表示更新（4.8 節）

一方、シリアル受信モードでは追加要素として、複数ポート受信が可能であり、同期タイムライン生成（同期解析の場合）を行う（??節）。

4.4 ファイル解析モードの設計・実装

4.4.1 ファイル解析モードの概要

ファイル解析モードは、単一ノードの動作を対象として、取得済みログの解析と可視化を行うモードである。ユーザは実行ログファイルと状態遷移図 JSON ファイルをアップロードし、ログ解析によって得られた状態列 *states* と変数値スナップショット列 *vars* を用いて、状態遷移図のハイライト表示および変数ウォッチ表示を行う。

4.4.2 ファイル解析モードの UI 設計

ファイル解析モードの UI は図 4.2 に示すように、主に以下の要素から構成される。

- (1) 実行ログファイルアップローダ
- (2) 状態遷移図 JSON ファイルアップローダ
- (3) ステップ実行制御ボタン（次へ／前へ／初期へ）
- (4) スライダーによるステップジャンプ
- (5) 変数ウォッチリスト表示エリア
- (6) 状態遷移ログ表示エリア
- (7) 状態遷移図表示エリア

ユーザは、(1)(2) により入力ファイルを選択、アップロードする。実行ログは解析され、状態遷移と変数スナップショットとして整理され、ステップ再現に用いられる。状態遷移図 JSON は状態遷移図の描画に用いられる。ユーザは (3)(4) によりステップ番号 i を操作し、(5) の変数値、(6) の状態遷移ログ、(7) の状態遷移図ハイライトを同一ステップに基づいて確認できる。

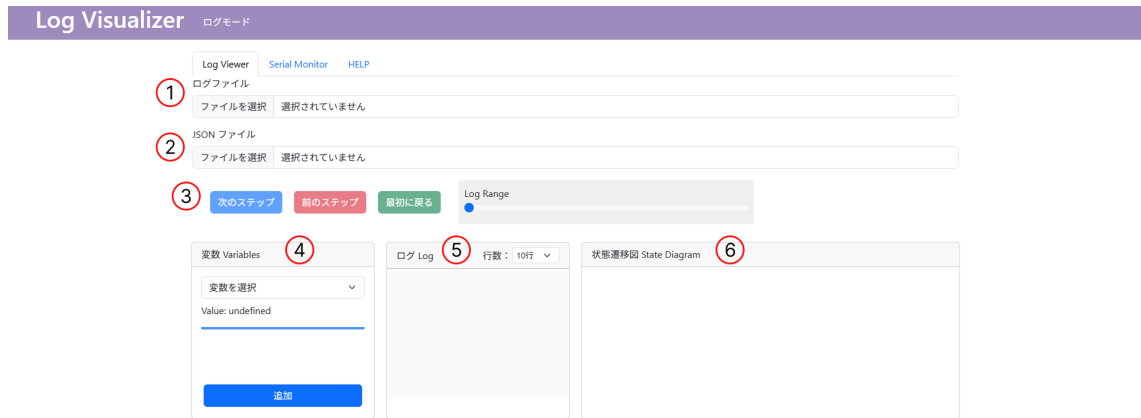


図 4.2: ファイル解析モードのレイアウト

4.5 シリアル受信モードの設計・実装

4.5.1 シリアル受信モードの概要

シリアル受信モードは、組込みデバイスを PC に接続し、シリアルポートからログを受信し、シリアルモニタ上に表示するモードである。受信ログはデバイスごとに蓄積し、ノード数と目的に応じて、**単一ノード解析**、**同期解析 (2 台)**、**マルチノード収集 (3 台以上)** をシステム側で自動的に切り替える。

単一ノード解析では、対象の 1 台のノードの受信ログを解析して *states* と *vars* を構築し、ファイル解析モードと同様に可視化デバッグを行う。同期解析では、2 台の受信ログに受信時刻を付与し、時系列を統合した同期タイムラインを生成した上で、同一時点における両ノードの状態と変数値を比較できる。マルチノード収集では、3 台以上のデバイスからログを受信し、ログの保存やコピーを主目的として扱う。(1/22 未実装)

4.5.2 シリアル受信モードのUI設計

シリアル受信モードは、ログ受信のためのシリアルモニタと、受信ログに基づいて可視化デバッグを行うUIの2つで構成される。二つのUIは同一画面内に配置され、シリアルモニタで受信したログをそのまま可視化デバッグへ引き継げるように設計されている。

シリアルモニタのUI

シリアルモニタのUIは図4.3に示すように、主に以下の要素から構成される。

- ポート選択ドロップダウン
- 接続ボタンおよび切断ボタン
- シリアルモニタ表示領域
- ログ解析ボタン（単一ノード解析用）
- 同期解析ボタン（同期タイムライン生成）

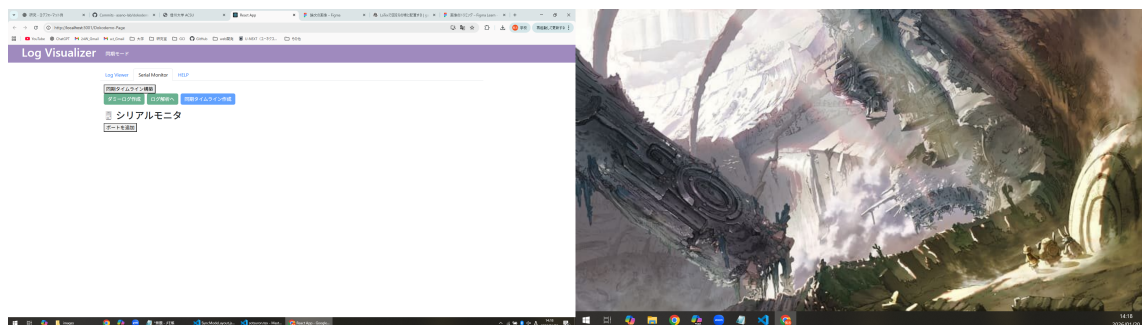


図 4.3: シリアルモニタのレイアウト

ユーザはPCと組み込みデバイスをUSB接続し、ポート選択ドロップダウンから接続するシリアルポートを選択する（図4.4）。接続ボタン押下によりログの読み取りを開始し、切断ボタン押下により読み取りを中断する。接続中は各デバイスから送信されるログをリアルタイムに受信し、シリアルモニタ表示領域に逐次表示する。



図 4.4: シリアルポート選択ドロップダウン

単一ノード解析では「ログ解析」ボタンを押下することで対象ノードの解析を実行する。同期解析では「同期解析」ボタンを押下することで2台分のログを統合し、同期タイムラインを生成する。生成された結果は可視化デバッグUIに反映され、ステップ番号 i に基づく逐次確認が可能となる。マルチノード収集では、ログ解析や同期解析は行わず、受信ログの保存・コピーを主目的とする（1/22 未実装）。

可視化デバッグのUI

シリアル受信モードの可視化デバッグUIは図4.5に示す。同期解析では、ノード1とノード2の状態遷移図表示と変数ウォッチ表示を左右に配置し、中央に同期タイムライン（統合された状態遷移ログ）を表示する。単一ノード解析では、ファイル解析モードと同じレイアウトになるようにした。

本UIでは、ステップ操作ボタンとスライダーを共通入力として、ステップ番号 i に応じて両ノードの表示を同時に更新する。これにより、ある時点における両ノードの状態と変数値を同一基準で比較できる。また、状態遷移図の描画は各ノードごとに独立して行われるため、それぞれのノードに応じた状態遷移図JSONをアップロードする必要がある。

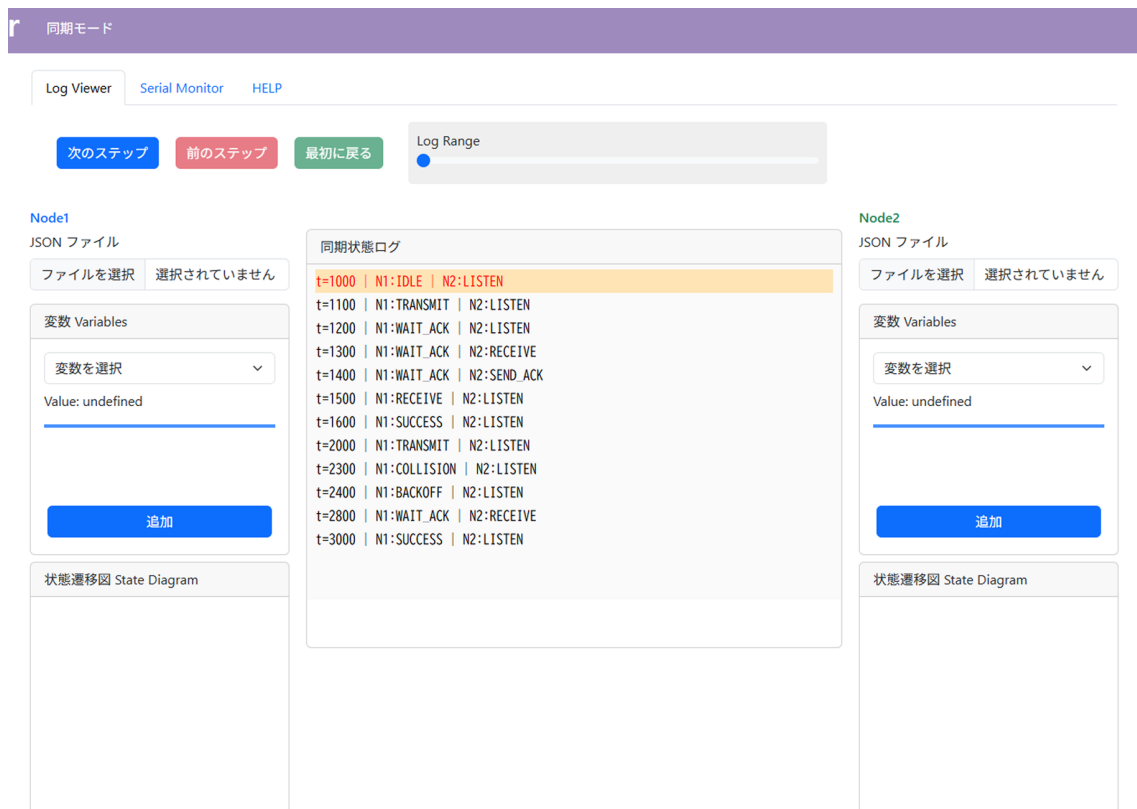


図 4.5: 同期モードのレイアウト

4.5.3 ブラウザ上でのシリアルモニタ実装

シリアル受信モードでは、専用のデスクトップアプリケーションを用いず、ブラウザ上でシリアル通信を扱えるように設計している。具体的には Web Serial API を用いることで、Web ブラウザから直接シリアルポートへ接続し、組み込みデバイスのログを受信する。ユーザは UI からポートを選択し、接続／切断を制御する。接続中は各デバイスから送信されるログをリアルタイムに受信し、逐次表示できる。なお、本方式はブラウザ環境に依存するため、利用環境の制約（対応ブラウザ・権限操作等）が存在する。

4.5.4 同期タイムラインの生成

同期タイムライン生成では、各デバイスから受信したログ行に対して受信時刻を基準としたタイムスタンプを付与する。状態遷移ログおよび変数更新ログはイベントとして扱い、デバイスごとに時系列順に管理する。その後、複数デバイスのイベント列を時刻

情報に基づいてマージし、共通の時系列イベント列を構築する。さらに、この統合イベント列から、各時刻における各デバイスの状態スナップショットを再構成し、同期タイムラインとして可視化する。

図 4.6 および図 4.7 に、同期モードにおけるタイムライン表示例を示す。

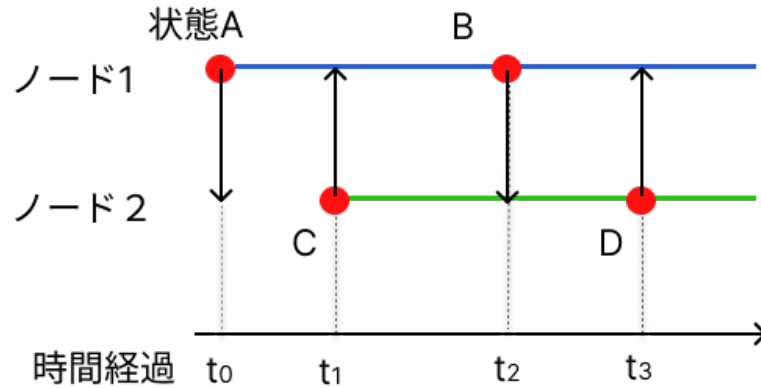


図 4.6: 同期モードのタイムライン (例 1)

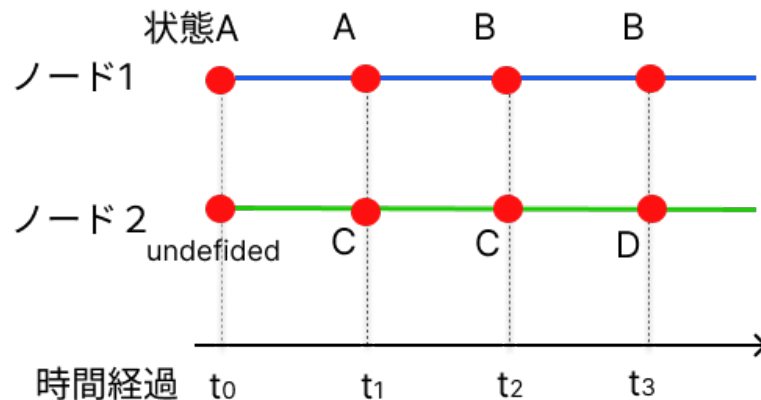


図 4.7: 同期モードのタイムライン (例 2)

4.6 ログ設計

4.6.1 ログ生成の設計

本節では、組込みデバイスから生成されるログの設計について述べる。ログは、状態遷移イベント、変数変更イベント、およびその他ログから構成される。デバッグ支援システムは、状態遷移イベントと変数変更イベントのみを解析対象とし、その他ログは解析時に無視する。

状態遷移イベント

状態遷移イベントは、組込みシステムの実行中に状態が遷移した際に生成されるログであり、STATE: 状態名 の形式で表現される。状態が切り替わるたびにファームウェア内の `updateState()` 関数が呼び出され、現在の状態をログ出力する設計とした。これにより、デバッグ支援システムは受信ログから現在の状態を把握し、状態遷移図上で該当状態をハイライト表示できる。

変数変更イベント

変数変更イベントは、組込みシステム内の変数が変更された際に生成されるログであり、[VAR:<name>=<value>] の形式で表現される。変数出力にはC++のマクロ機構を用い、開発者は状態遷移図における各状態の entry 内に `PRINT_VAR_LOG(変数)` と記述するだけで、変数名と値を統一形式で出力できる。これにより、デバッグ用コードの記述量を抑えつつ出力形式を統一でき、デバッグ支援システム側での解析と変数追跡を容易にする。

その他ログ

その他ログは、デバッグ情報やエラーメッセージなど、状態遷移イベントおよび変数変更イベント以外の情報を含む。これらは実行状況把握やトラブルシューティングの補助として有用であるが、本システムの状態・変数解析の対象とはしないため、解析時には無視される。

ソースコード 4.1: 生成されるログの例

```
1 STATE: LISTEN
2 [VAR: var01=0]
3 [VAR: var02=10]
4 STATE: LISTEN
5 STATE: RECEIVE
6 [VAR: var01=1]
7 [VAR: var02=11]
```

8	STATE: TRANSMIT
9	[VAR: var01=2]
10	[VAR: var02=100]

4.7 ログ解析の設計（共通処理）

4.7.1 概要（解析の目的と処理の流れ）

本システムのログ解析は、ログ列を入力として状態遷移をステップ単位で再構成し、デバッグ表示に必要な内部情報（状態名と変数値）を整形する処理である。処理は「ログを1行ずつ読み取り、行種別（状態遷移／変数更新／その他）に応じて内部データを更新する」流れで構成される。図4.8にログ解析全体の処理フローを示す。

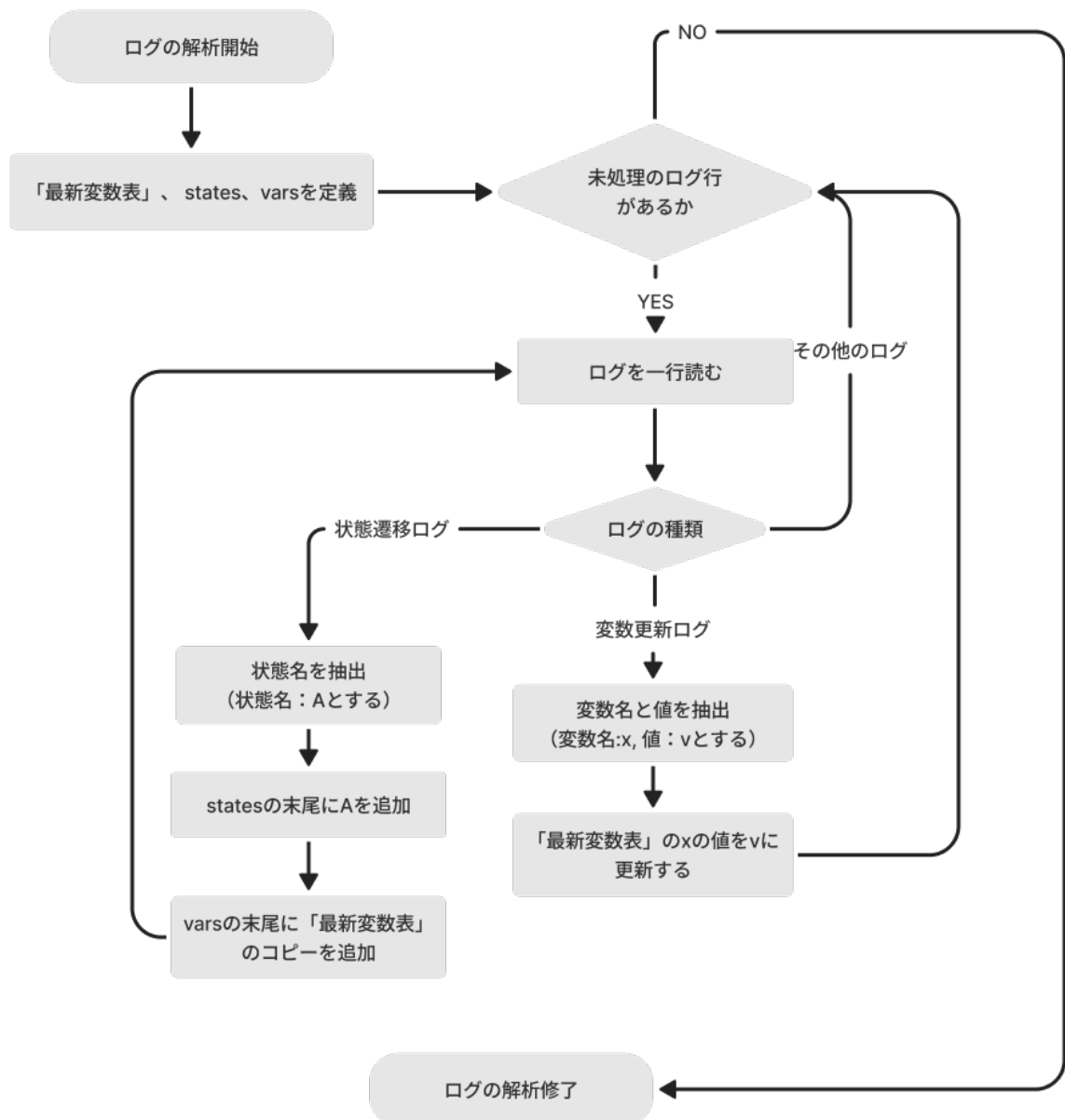


図 4.8: ログ解析の処理フロー

4.7.2 入力ログ形式とログ行の分類

ログは行単位のテキストとして扱い、各行は先頭文字列に基づいて以下の3種類に分類される。

- 状態遷移ログ： STATE: で始まる行
- 変数更新ログ： [VAR: で始まる行
- その他ログ：上記のいずれにも該当しない行

状態遷移ログと変数更新ログのみを解析対象とし、デバッグメッセージやエラーメッセージ等のその他ログは無視する。

4.7.3 解析結果として構築するデータ構造

本システムはログを「状態遷移単位（ステップ）」で整理するため、解析結果として (i) 状態列 *states* と (ii) 変数値スナップショット列 *vars* を構築する。状態遷移ログの出現回数をステップ番号 *i* とみなし、*states*[*i*] に状態名を、*vars*[*i*] にその状態における変数値集合（連想配列）を格納する。

また、直近までに観測された変数値集合を保持する連想配列を「最新変数表」と呼ぶ（実装上は *lastVars* に相当する）。最新変数表は、後述する変数値スナップショットの再構成（継承）に用いる。

4.7.4 変数値スナップショットの再構成方針（継承と更新）

変数更新ログは「値が変更された変数のみ」を出力する設計であるため、各状態において全変数の値が毎回ログに現れるとは限らない。そこで本システムでは、各状態における変数値を次の規則で再構成する。

- 変数更新ログが出現した場合：最新変数表中の該当変数のみを更新する。
- 状態遷移ログが出現した場合：その時点の最新変数表をコピーし、当該ステップの変数値スナップショット *vars*[*i*] として確定する。

さらに、同一状態中に変数更新ログが続けて出現する場合があるため、直近ステップ *vars*[*i*] が既に生成されている状況では、最新変数表の更新内容を *vars*[*i*] にも反映する。これにより、「状態遷移直後に出力された変数更新」も、当該状態の変数値として取り込める。図 4.9 に、変数値の継承（コピー）と更新の関係を示す。

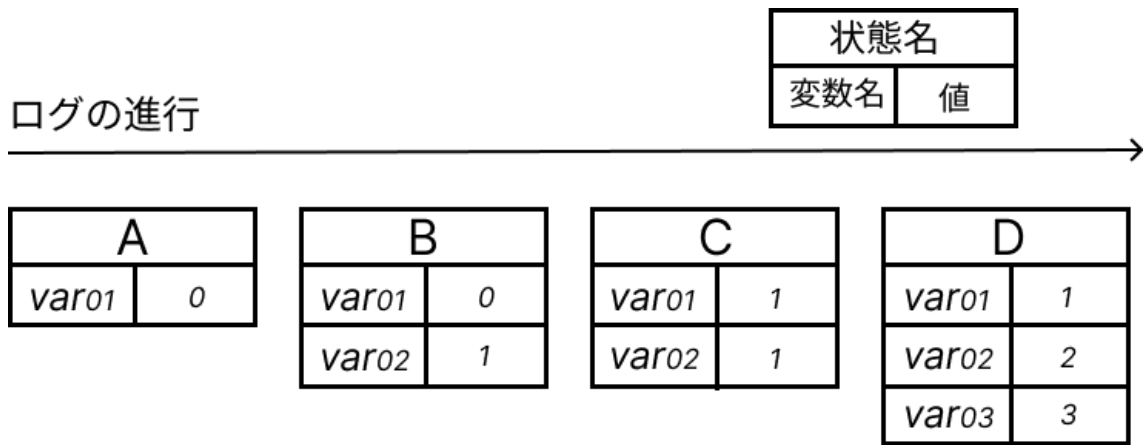


図 4.9: 変数値スナップショットの継承と更新

4.7.5 行処理アルゴリズム（擬似コード）

上記処理の擬似コードをソースコード 4.2 に示す。

ソースコード 4.2: ログ解析アルゴリズム（擬似コード）

```

1 states <- empty list
2 vars <- empty list
3 Vlatest <- empty map // 最新変数表（に相当）lastVars
4
5 for each line in log_lines:
6     if line is STATE:
7         s <- extract_state_name(line)
8         append(states, s)
9         append(vars, copy(Vlatest)) // スナップショット確定
10
11     else if line is VAR:
12         (x, v) <- extract_var_update(line)
13         Vlatest[x] <- v // 最新変数表を更新
14         if length(vars) > 0: // 直近ステップが存在する場合
15             vars[last_index][x] <- v // 直近スナップショットにも反映
16

```

```
17     else :  
18         continue                // その他ログは無視  
19  
20 return (states, vars)
```

4.7.6 解析結果例

ログ解析によって得られる *states* と *vars* の例をソースコード 4.3 に示す。この例は、図 4.9 に示した「継承と更新」の関係に対応している。

ソースコード 4.3: ログ解析結果の例 (*states* と *vars*)

```
1 states = ["A", "B", "C", "D"]  
2  
3 vars = [  
4     { var01: 0 },                // A のスナップショット  
5     { var01: 0, var02: 1 },      // B のスナップショット  
6     { var01: 1, var02: 1 },      // C のスナップショット  
7     { var01: 1, var02: 2, var03: 3 } // D のスナップショット  
8 ]
```

4.8 ステップ番号に基づく共通表示ロジック

4.8.1 共通の前提：ステップ番号 i と状態スナップショット

本システムでは、ログ解析により構築された状態列 *states*[i] と変数値スナップショット列 *vars*[i] (4.7 節) を基盤として、各種表示を統一的に制御する。ここで i は「状態遷移ログの出現順」に対応するステップ番号であり、ユーザがスライダー操作やボタン操作により i を変更すると、UI は *states*[i] および *vars*[i] を参照して表示を更新する。この設計により、ログモードと同期モードで入力経路が異なる場合でも、「ステップ番号 i を中心に表示を駆動する」という共通構造を維持できる。

4.8.2 ステップ実行のロジック

ステップ実行機能は、ステップ番号 i を1ずつ増減させることで、状態遷移の進行を逐次確認できるようにする機能である。ユーザ操作として、(1) 次のステップへ、(2) 前のステップへ、(3) 初期ステップへ、およびスライダーによる任意ステップへの移動を提供する。

内部的には現在のステップ番号を i とし、 $0 \leq i \leq (N - 1)$ (N は状態列の長さ) を満たすよう境界条件を設ける。ユーザ操作により i が更新されると、状態遷移ログ表示、変数ウォッチ表示、状態遷移図のハイライト表示を同時に更新する。

ソースコード 4.4: ステップ実行の擬似コード

```
1 # i: currentIndex, N: length(states)
2
3 onClickNext():
4     if i < N-1: i ← i + 1
5
6 onClickPrev():
7     if i > 0: i ← i - 1
8
9 onClickInitial():
10    i ← 0
11
12 onSliderChange(newI):
13    i ← clamp(newI, 0, N-1)
14
15 # i changes => update all views
16 render():
17     currentState ← states[i]
18     currentVars  ← vars[i]
```

4.8.3 変数ウォッチ機能のロジック

変数ウォッチ機能は、ユーザが監視したい変数（ウォッチリスト）を指定し、現在ステップ i における変数値 $vars[i]$ を参照して表示する機能である。ログは「変更された変数のみ」が出力されるが、ログ解析段階で変数値スナップショットを再構成しているため（4.7節）、表示側では単純に $vars[i]$ を参照するだけでよい。

ウォッチリストを $W = \{w_1, w_2, \dots\}$ とすると、各変数 w に対して $vars[i][w]$ を表示する。当該ステップで一度も観測されていない変数は未定義となり得るため、その場合は “—” などのプレースホルダ表示とする。

ソースコード 4.5: 変数ウォッチの擬似コード

```

1 #W: watch list (variable names)
2 # vars[i]: map of variable→value
3
4 renderWatch(i):
5   for each w in W:
6     if w in vars[i]:
7       show(w, vars[i][w])
8     else:
9       show(w, "—")

```

4.8.4 状態遷移表示機能（ハイライト表示）

状態遷移表示機能は、ステップ i に対応する現在状態 $states[i]$ を状態遷移図上でハイライトし、状態遷移の進行を視覚的に提示する機能である。ハイライトの色や強調表現の細部は割愛するが、基本的には「現在状態」と「直前状態」の関係に基づいて表示を変化させる。

具体的には、 $s_i = states[i]$ 、 $s_{i-1} = states[i-1]$ とし、 $s_{i-1} = s_i$ の場合は「同一状態の継続」とみなして強調表示を変更するなど、状態遷移の有無を表示上で区別する。

ソースコード 4.6: ハイライト判定の擬似コード

```
1 s <- states[i]
2 sp <- (i>0) ? states[i-1] : null
3
4 if sp == s:
5   highlight(s, "stay") # 同一状態継続
6 else:
7   highlight(s, "move") # 遷移先状態
```

4.8.5 Mermaid による状態遷移図描画（構文生成ロジック）

状態遷移図の描画には Mermaid.js を用いる。Mermaid は状態遷移図をテキスト（構文）として記述し、その文字列から図を生成するため、本システムでは状態遷移図 JSON（状態集合・遷移集合・初期状態）から Mermaid 構文文字列を生成する。さらに、ステップ i に応じて現在状態をハイライトするスタイル指定を付加することで、ログと状態遷移図を連動させた表示を実現する。

構文生成は概ね次の手順で行う。(1) Mermaid のヘッダ（stateDiagram-v2 等）を出力、(2) 初期状態への遷移を出力、(3) 遷移集合を走査して矢印（from --> to）を列挙、(4) 現在状態 $states[i]$ に対するスタイル指定を付加、である。自己ループなど表示上不要な遷移は除外する。

ソースコード 4.7: Mermaid 構文生成の擬似コード

```
1 buildMermaid(stateJson, states, i):
2   M <- "stateDiagram-v2\n"
3   M <- M + "direction TB\n"
4   M <- M + "[*] --> " + stateJson.initialState + "\n"
5
6   for each (from,to) in stateJson.transitions:
7     if from != null and to != null and from != to:
8       M <- M + from + " --> " + to + "\n"
9
```

```
10   s <- states[i]
11   sp <- (i>0) ? states[i-1] : null
12
13   if sp == s:
14     M<- M+ "style " + s + " ... stay-style ...\n"
15   else:
16     M<- M+ "style " + s + " ... move-style ...\n"
17
18   return M
```

以上のように、本システムではステップ番号 i を中心に、(1) 状態列 $states[i]$ と (2) 変数スナップショット $vars[i]$ を参照して、各表示（ログ表示・ウォッチ表示・状態遷移図表示）を一貫して更新する構造としている。

第 5 章

評価

5.1 評価の目的と方針

5.1.1 評価の目的（何を確かめたいか）

本章では、本研究で開発したデバッグ支援システムについて、提案機能が実際の実機ログ解析において有効に機能することを確認する。特に、(1) 状態遷移の追跡、(2) 変数値追跡（ウォッチ）、(3) ステップ実行に基づく逐次確認、および (4)（同期モードにおける）複数ノードの比較可能性、の観点から、実ログを用いて動作例を示し、定性的に評価する。

5.1.2 評価対象

評価対象は、ログ解析結果を用いてデバッグ表示を行う 2 つの動作モードである。ただし、本研究では「ログファイルを入力として行うデバッグ」と「単一ノードをシリアルモニタ経由で受信して行うデバッグ」は、ユーザが行う確認作業および画面上の操作がほぼ共通である。そこで本章では、単一ノードを対象とした評価を「単一ノードデバッグ」として共通化し、そのうえで同期モード固有の機能（同期タイムラインによる比較）を別途評価する構成とする。

- 単一ノードデバッグ（共通部分）
 - － 状態遷移の可視化（状態遷移図のハイライトとログ表示）

- 変数ウォッチ機能（状態ステップに対応した変数値追跡）
- ステップ実行および任意ステップへの移動（スライダ）
- 同期モード固有機能（差分）
 - 2 台のノードログを統合した同期タイムライン表示
 - 同一時刻における状態・変数の比較

5.1.3 評価で用いるデータ（実験対象）

評価には、実機で動作する無線通信プロトコルの実行ログと、対応する状態遷移図（JSON）を用いる。本研究ではログ設計として状態遷移ログ（STATE:）と変数更新ログ（[VAR:）を用いており、評価においても同形式のログを入力として用いる。また、同期モードの評価では 2 台のデバイスから得られたログを用いる。

5.2 単一ノードデバッグの評価（ログモード／単一ノード受信の共通部分）

5.2.1 評価シナリオ

本節では、単一ノードの実行ログ（.log ファイル）を入力として、状態遷移の進行および変数値の推移をステップ単位で追跡できることを確認する。単一ノードの場合、入力方法はログファイルアップロードであってもシリアルモニタ受信であっても、解析結果として構築される状態列 *states* と変数スナップショット列 *vars* を基に表示が行われるため、本節では両者に共通するデバッグ作業として記述する。

また、本システムでは状態遷移図を表示するために、状態遷移図 JSON を入力として Mermaid 構文へ変換し、その描画結果を GUI に表示する。したがって、本節では読み手の理解の流れに合わせて、(1) ログ入力による「状態遷移ログ表示」と「変数ウォッチ表示」、(2) JSON 入力による「状態遷移図描画」、(3) それらを同一ステップで同期させる「ステップ操作・スライダ操作」の順に評価内容を述べる。

5.2.2 ログ入力と解析結果の生成 (states/vars)

評価に用いる入力ログの抜粋を 5.1 に示す。本ログは、状態遷移ログ (STATE:) と変数更新ログ ([VAR:)] から構成され、待機状態の継続 (同一状態の連続出現) やパケットの送受信、衝突、バックオフを含む典型的な遷移を表している。ログ入力後、システムはログ解析を行い、状態列 *states* と、各ステップに対応する変数スナップショット列 *vars* を構築し、以降の表示更新はこれらの配列インデックス (ステップ番号) に基づいて行われる。

ソースコード 5.1: デバッグシステムに入力するログ (抜粋)

```
1 STATE: IDLE
2 [VAR:DEVICE_EI=1]
3 STATE: TRANSMIT
4 [VAR:success_cnt=0]
5 STATE: WAITACK
6 STATE: WAITACK
7 STATE: RECEIVE
8 [VAR:ack=1]
9 [VAR:success_cnt=1]
10 STATE: SUCCESS
11 STATE: IDLE
12 [VAR:DEVICE_EI=1]
13 STATE: TRANSMIT
14 [VAR:success_cnt=1]
15 STATE: WAITACK
16 STATE: WAITACK
17 STATE: WAITACK
18 STATE: COLLISION
19 STATE: BACKOFF
20 STATE: BACKOFF
21 STATE: TRANSMIT
```

```
22 [VAR: success_cnt=1]
23 STATE: WAITACK
24 STATE: RECEIVE
25 [VAR: ack=1]
26 [VAR: success_cnt=2]
27 STATE: SUCCESS
```

ログ解析により得られる状態列の抜粋をソースコード 5.2 に示す。同一状態が連続する場合も、ログ出現順に状態名が列として保持されるため、GUI 側では当該インデックスを「現在ステップ」として扱い、待機の継続等をステップ単位で追跡できる。

ソースコード 5.2: ログ解析結果の例 (states の抜粋)

```
1 states = [
2     "IDLE",
3     "TRANSMIT",
4     "WAITACK",
5     "WAITACK",
6     "RECEIVE",
7     "SUCCESS",
8     "IDLE",
9     "TRANSMIT",
10    "WAITACK",
11    "WAITACK",
12    "WAITACK",
13    "COLLISION",
14    "BACKOFF",
15    "BACKOFF",
16    "TRANSMIT",
17    "WAITACK",
```



```

18  "RECEIVE",
19  "SUCCESS",
20 ]

```

変数スナップショット列 *vars* の抜粋をソースコード 5.3 に示す。本研究のログ設計では、[VAR:...] は「変更された変数のみ」を出力するため、ログに現れない変数については直前値を継承し、各ステップで参照可能な形に再構成する（節参照）。この再構成により、状態遷移 (*states*) と変数値 (*vars*) を同一ステップに対応付けて表示できる。ソースコード 5.2 の *i* 行目の状態（ステップ）に対応する変数スナップショットはソースコード 5.3 の *i* 行目に対応している。

ソースコード 5.3: ログ解析結果の例 (*vars* の抜粋)

```

1  vars = [
2    {"DEVICE_ID": "1"},
3    {"DEVICE_ID": "1", "success_cnt": "0"},
4    {"DEVICE_ID": "1", "success_cnt": "0"},
5    {"DEVICE_ID": "1", "success_cnt": "0"},
6    {"DEVICE_ID": "1", "success_cnt": "1", "ack": "1"},
7    {"DEVICE_ID": "1", "success_cnt": "1", "ack": "1"},
8    {"DEVICE_ID": "1", "success_cnt": "1", "ack": "1"},
9    {"DEVICE_ID": "1", "success_cnt": "1", "ack": "1"},
10   {"DEVICE_ID": "1", "success_cnt": "1", "ack": "1"},
11   {"DEVICE_ID": "1", "success_cnt": "1", "ack": "1"},
12   {"DEVICE_ID": "1", "success_cnt": "1", "ack": "1"},
13   {"DEVICE_ID": "1", "success_cnt": "1", "ack": "1"},
14   {"DEVICE_ID": "1", "success_cnt": "1", "ack": "1"},
15   {"DEVICE_ID": "1", "success_cnt": "1", "ack": "1"},
16   {"DEVICE_ID": "1", "success_cnt": "1", "ack": "1"},
17   {"DEVICE_ID": "1", "success_cnt": "1", "ack": "1"},
18   {"DEVICE_ID": "1", "success_cnt": "2", "ack": "1"},
19   {"DEVICE_ID": "1", "success_cnt": "2", "ack": "1"}

```

20]

5.2.3 状態遷移ログ表示（ログ入力の可視化）

ログ表示領域では、ログ解析により構築された状態列 *states* (5.2) を基に、状態遷移を一定行数で表示する。ユーザは表示行数として 10 行、30 行、50 行、100 行を選択でき、選択した行数に応じて俯瞰と詳細確認を切り替えられる。図 5.1 に表示例を示す。

また、連続して同一状態が出現する場合（自己遷移や待機状態の継続）においても、ステップ番号（インデックス）の進行に従って、状態列 *states* の該当要素がログ上の現在位置として反映される。これにより、「同じ状態が続いている」ことをステップ単位で追跡できることを確認した。

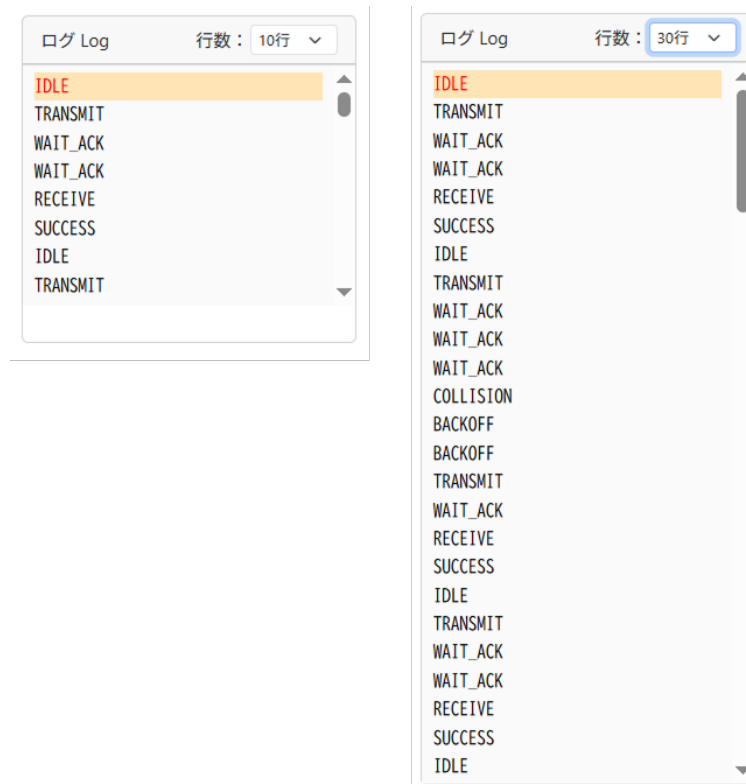


図 5.1: 状態遷移ログ表示（左：10 行表示、右：30 行表示）

5.2.4 変数ウォッチ表示（状態ステップに対応した変数値追跡）

変数ウォッチ領域では、ユーザが監視対象とする変数を選択し（図 5.2）、選択された

変数の値をステップ番号に対応付けて表示する（図 5.3）。この表示は、ログ解析で生成される変数スナップショット列 *vars*（5.3）を参照して行われ、各ステップにおける変数値を一貫した形式で確認できる。

本研究のログ設計では、[VAR:...] が「変更された変数のみ」を出力するため、ログ上で値が現れないステップにおいても直前値を継承することで、各ステップのスナップショットを再構成している。その結果、ユーザはログ行を手作業で追跡・照合することなく、状態列 *states* と同一ステップ上で変数値の推移を追跡できることを確認した。

以上より、本機能により「状態遷移の進行」と「変数値の変化」を同一ステップで対応付けられ、デバッグ時の確認作業を簡略化できることを示した。



図 5.2: 変数ウォッチ表示（変数選択）

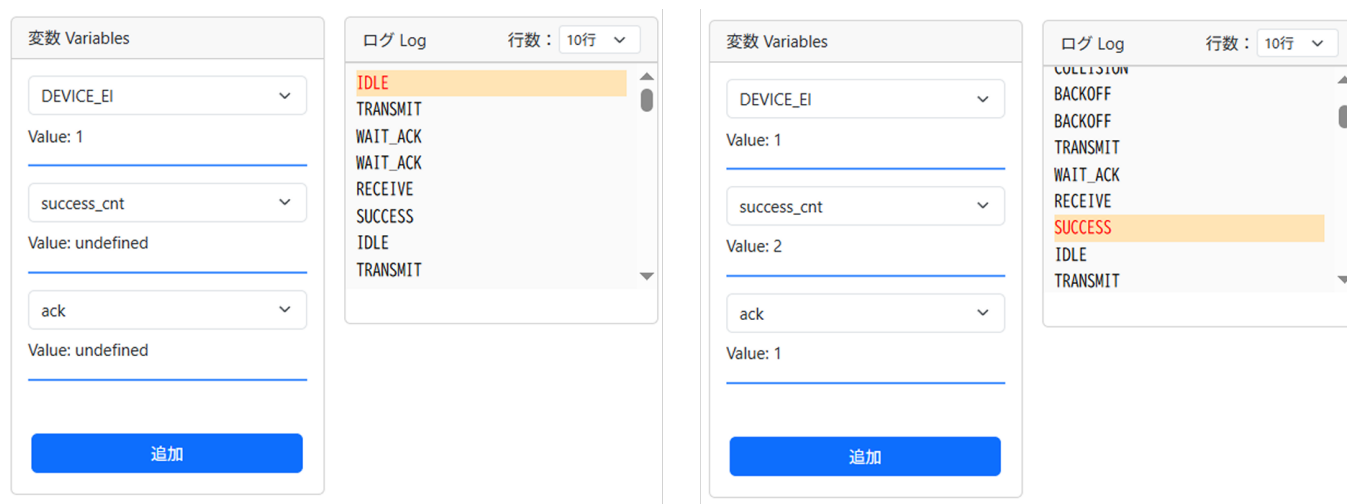


図 5.3: 変数ウォッチ表示（ステップによる変数の変化）

5.2.5 状態遷移図表示（JSON 入力→Mermaid 変換→描画）

本研究で用いるプロトコルは、状態遷移図ベースであり、その可視化はデバッグにおいて重要である。状態遷移図は、状態遷移図 JSON を入力として Mermaid 構文（Mermaid が状態遷移図を描画するための記法）へ変換し、その結果を描画して表示する。

Pure ALOHA の状態遷移図 JSON 例をソースコード 5.4 に示す。states 配列に状態名を、transitions 配列に遷移関係を、initialState に初期状態を定義している。この JSON から生成された Mermaid 構文をソースコード 5.5 に示す。最下行で現在状態のスタイル指定（ハイライト）を付加している。

ソースコード 5.4: 状態遷移図 JSON の例（Pure ALOHA）

```
1 {
2   "states": [
3     "IDLE",
4     "TRANSMIT",
5     "WAITACK",
6     "SUCCESS",
7     "COLLISION",
8     "BACKOFF"
9   ],
10  "transitions": [
11    {"from": "IDLE", "to": "TRANSMIT"},
12    {"from": "TRANSMIT", "to": "WAITACK"},
13    {"from": "WAITACK", "to": "SUCCESS"},
14    {"from": "WAITACK", "to": "COLLISION"},
15    {"from": "WAITACK", "to": "WAITACK"},
16    {"from": "SUCCESS", "to": "IDLE"},
17    {"from": "COLLISION", "to": "BACKOFF"},
18    {"from": "BACKOFF", "to": "TRANSMIT"},
19    {"from": "BACKOFF", "to": "BACKOFF"}
20  ],
```

```
21   "initialState": "IDLE"  
22 }
```

ソースコード 5.5: 生成された Mermaid 構文 (Pure ALOHA)

```
1 stateDiagram-v2  
2 direction TB  
3 [*] --> IDLE  
4 IDLE --> TRANSMIT  
5 TRANSMIT --> WAITACK  
6 WAITACK --> SUCCESS  
7 WAITACK --> COLLISION  
8 SUCCESS --> IDLE  
9 COLLISION --> BACKOFF  
10 BACKOFF --> TRANSMIT  
11 style IDLE fill:#f9f,stroke:#333,stroke-width:2px;
```

図 5.4 に、Mermaid による状態遷移図の描画例を示す。状態遷移図が表示されることで、ログの状態列 (5.2) が図上の状態遷移として対応付けて理解しやすくなる。

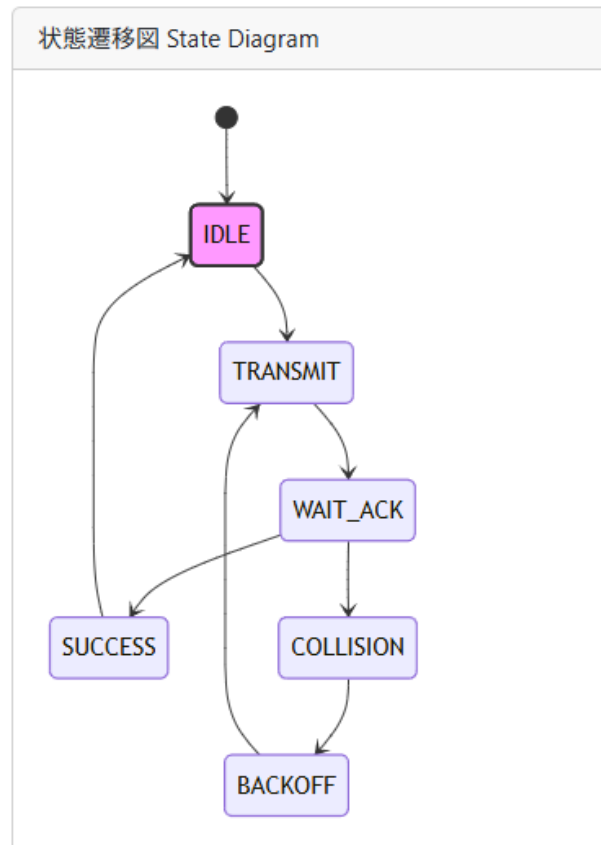


図 5.4: Mermaid による状態遷移図の描画例

5.2.6 ステップ操作・スライダ操作（表示要素の同期更新）

次ステップボタン・前ステップボタンの操作により、状態遷移図表示、状態遷移ログ表示、変数ウォッチ表示が、同一のステップ番号（インデックス）に基づいて同期して更新されることを確認する。図 5.5 に、ステップ実行により表示要素が連動して更新される例を示す。

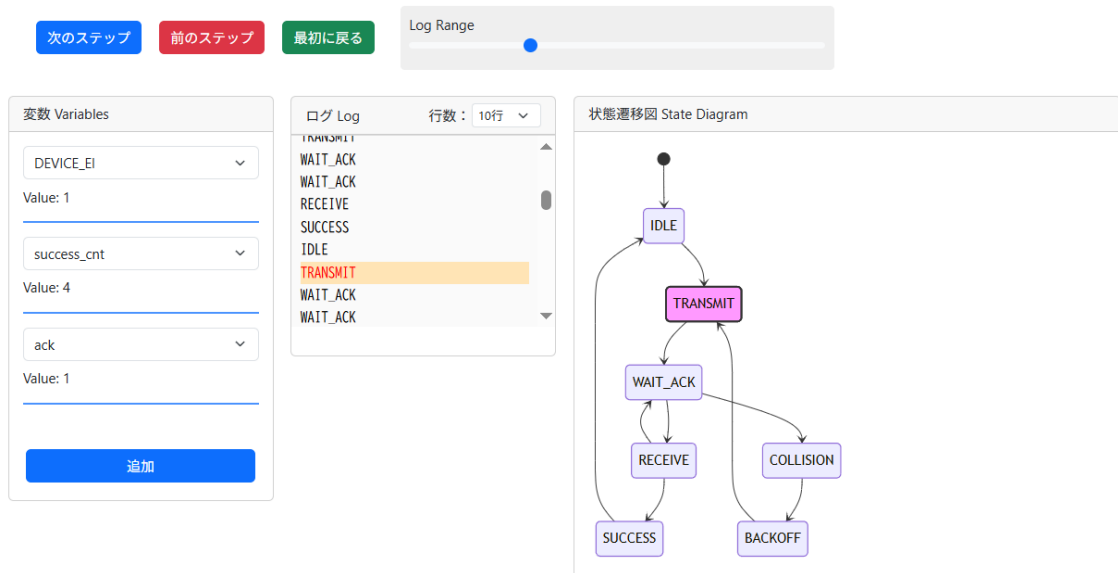


図 5.5: ステップ実行の様子

また、スライダ操作により任意ステップへ移動した場合も同様に、状態列 *states* (5.2) および変数スナップショット列 *vars* (5.3) の同一インデックスを参照することで、各表示要素が整合して更新されることを確認した。この一貫したインデックス参照により、「状態遷移図」「ログ」「変数」の対応関係を崩さずに任意位置へジャンプでき、逐次デバッグの効率化に寄与する。

5.3 同期モード固有機能の評価（2 ノード同期タイムライン）

5.3.1 評価シナリオ

本節では、同期デバッグモードの機能である「複数デバイスログの統合」と「同一時刻における状態・変数の同時参照」が実現できていることを、2 ノード構成（中継器 1 台＋端末 1 台）で確認する。具体的には、(i) シリアルモニタが各デバイスから状態遷移ログおよび変数ログを継続的に受信できること、(ii) 受信ログを時刻情報に基づき統合して同期タイムラインを構築できること、(iii) タイムライン上の任意時刻において両ノードの状態・変数が対応付けて提示され、送受信関係や待機の重なり等の時間的關係が読み取れること、の 3 点を評価項目とする。なお、実機ログは電波環境や実行タイミングにより再現性が低く、論文において同一条件の画面例を提示しにくい。そのため、本節の図

示では実機実行で得られるログ形式に一致する疑似ログを投入し、代表例として再現性のあるスクリーンショットを示す。

5.3.2 シリアルモニタによるログ受信

まず、シリアルモニタが各デバイスからのログをデバイス別に分離して表示できることを確認する。図 5.6 は、中継器（左）と端末（右）からのログが同時に流入する状況で、状態遷移（STATE: ...）と変数出力（<VAR: key=value>）が時刻付きで蓄積される様子を示す。ここで重要なのは、後続の同期処理に必要な「時刻」「イベント種別（状態/変数）」「値」が欠落なく、2 ノード並行して取得できている点である。従来研究では、Visual Studio Code の PlatformIO 拡張機能内のシリアルモニタを用いて同様のログ受信を行っていた。この場合、複数の組み込みデバイスのログを並行して取得することができなかったのが、デバッグ効率を下げる一つの要因となっていた。本システムでは Web ベースのシリアルモニタを独自に実装し、複数ノードからのログを同時に受信・表示でき、そのログをそのままデバッグへ移行できる点が優れていると言える。

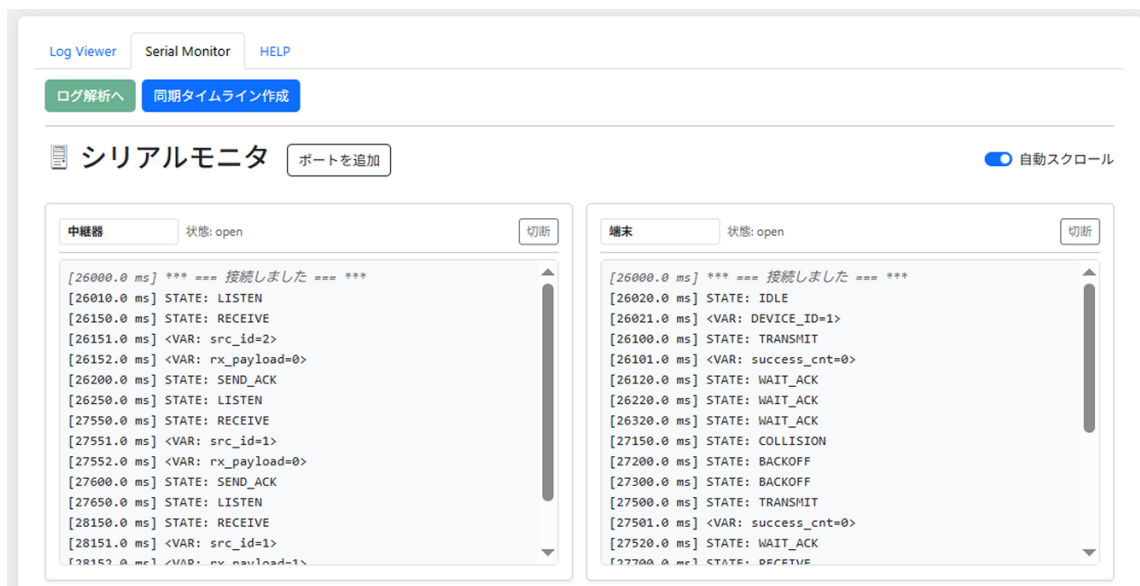


図 5.6: シリアルモニタによるログ受信の様子（左：中継器、右：端末）

5.3.3 同期タイムライン生成と表示

次に、受信ログを統合して同期タイムラインとして再構成できることを確認する。同期タイムラインは、各ノードで観測されたイベント列を時刻順に統合し、共通の時間軸

上で「ある時刻における各ノードの状態と変数値スナップショット」を同時に保持するデータ構造として生成される（4.5.4節参照）。シリアルモニタでログを受信した後、ユーザが「同期タイムライン生成」ボタンを押下することで、ログ解析とタイムライン構築が実行される。ソースコード 5.6 に、生成された同期タイムラインの例を示す。4.5.4 節で紹介したように、タイムラインは各時刻 t における各ノードの状態と変数値を含むオブジェクトの配列として表現される。

図 5.7 に、生成された同期化されたの状態遷移表示例を示す。本表示により、中継器と端末のイベントが単一の時系列として整列され、時刻 t に対応する両ノードの状態が同時に提示されることが分かる。変数ウォッチ機能については、単一ノードの場合と同様のため、ここでは省略する。これにより、2 台のノードが同時点でどのような状態にあり、どのような変数値を持っているかを一目で把握できる。

ソースコード 5.6: 生成された同期タイムライン（抜粋）

```
1 timeline = [  
2   {  
3     t: 1000,  
4     states: {  
5       Node1: { state: "LISTEN", vars: {} },  
6       Node2: { state: "IDLE", vars: { DEVICEID: 1 } },  
7     },  
8   },  
9   {  
10    t: 1100,  
11    states: {  
12      Node1: { state: "RECEIVE", vars: { src_id: 2, rx_payload: 0 } },  
13      Node2: { state: "TRANSMIT", vars: { DEVICEID: 1, success_cnt: 0 } },  
14    },  
15  },  
16  {  
17    t: 1200,
```

```

18     states: {
19         Node1: { state: "SENDACK", vars: { src_id: 2, rx_payload: 0 } }, // 端
           末へ2返し中ACK
20         Node2: { state: "WAITACK", vars: { DEVICEID: 1, success_cnt: 0 } },
21     },
22 },
23 {
24     t: 1300,
25     states: {
26         Node1: { state: "LISTEN", vars: { src_id: 2, rx_payload: 0 } },
27         Node2: { state: "WAITACK", vars: { DEVICEID: 1, success_cnt: 0 } },
28     },
29 }
30 ];

```

同期状態ログ		
t=1000	N1:LISTEN	N2:IDLE
t=1100	N1:RECEIVE	N2:TRANSMIT
t=1200	N1:SEND_ACK	N2:WAIT_ACK
t=1300	N1:LISTEN	N2:WAIT_ACK
t=2100	N1:LISTEN	N2:COLLISION
t=2200	N1:LISTEN	N2:BACKOFF
t=2400	N1:LISTEN	N2:TRANSMIT
t=2500	N1:RECEIVE	N2:WAIT_ACK
t=2600	N1:SEND_ACK	N2:WAIT_ACK
t=2700	N1:LISTEN	N2:RECEIVE
t=2800	N1:LISTEN	N2:SUCCESS
t=3000	N1:LISTEN	N2:TRANSMIT
t=3100	N1:RECEIVE	N2:WAIT_ACK
t=3200	N1:SEND_ACK	N2:WAIT_ACK

図 5.7: 同期化された状態遷移表示例

5.3.4 複数ノード比較の有効性（定性的観察）

最後に、同期タイムラインによって複数ノード間の相互作用を把握しやすくなる点を定性的に確認する。図5.8は、同期タイムライン上の特定時刻を選択した際に、両ノードの状

状態遷移図および変数表示が同時に更新される様子を示す。このとき、端末側が TRANSMIT から WAIT_ACK へ遷移している間に、中継器側で RECEIVE や SEND_ACK が発生しているか否かを同一画面で対応付けて確認できる。また、端末が WAIT_ACK を継続した後に COLLISION へ遷移する場合、中継器側に該当端末由来の受信（例：src_id=1）が存在しない、あるいは別端末由来の受信（例：src_id=2）が優先して観測される、といった関係を時系列として追跡できる。このように、同期タイムラインは「同一時刻の状態・変数の突き合わせ」を可能にし、単一ノードのログ閲覧では把握しにくい送受信の因果関係や待機時間の重なりを解釈しやすくする。

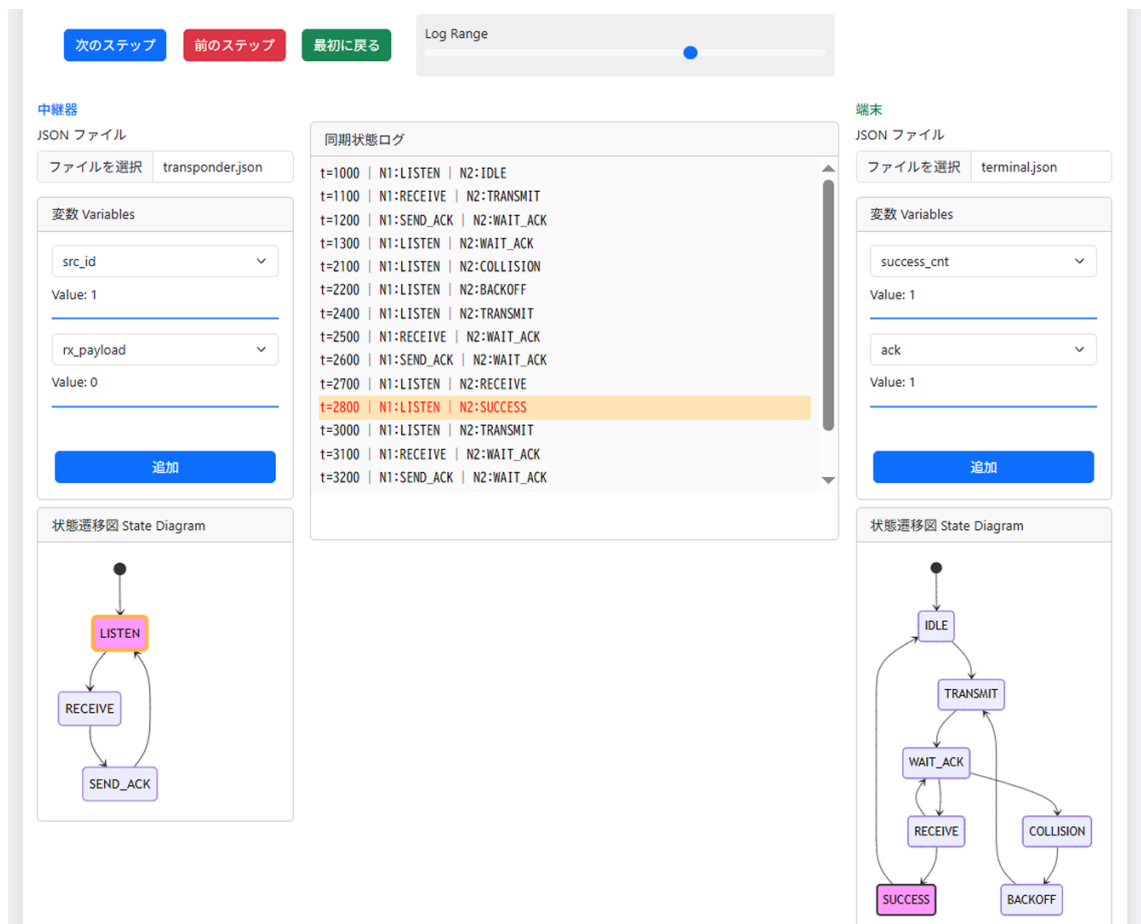


図 5.8: 同期デバッグ実行例（タイムライン選択に連動した複数ノードの同時表示）

5.4 定性的評価

5.4.1 単一ノードデバッグに対する効果

単一ノードデバッグでは、状態遷移をステップ単位で追跡し、各ステップにおける変数

スナップショットを提示できるため、ログ行を手作業で突き合わせる負担を軽減できる。さらに、状態遷移図・ログ・変数表示が同一インデックスで連動して更新されるため、実行の流れを視覚的に追跡しながら原因箇所を特定しやすい。

5.4.2 同期デバッグに対する効果

同期デバッグでは、複数ノードのログを共通時間軸に統合することで、同一時刻における状態・変数の比較が可能となる。これにより、送受信や ACK 待機の関係、待機状態の重なり、競合・再送といった分散実行特有の現象を、複数ノードの対応関係として把握できる。

5.4.3 まとめ

以上より、本研究で開発したデバッグ支援システムは、単一ノードに対しては状態遷移と変数値を統合的に可視化して追跡性を向上させ、複数ノードに対しては同期タイムラインにより同時刻比較を可能にすることで相互作用の理解を支援できることを確認した。

第 6 章

考察

6.1 有効性の要因

6.1.1 状態遷移単位への再構成

本システムでは、ログを 1 行ずつ処理し、状態遷移ログの出現をステップ境界として扱うことで、実行を「状態遷移単位」に再構成した。これにより、ログ表示、状態遷移図ハイライト、変数ウォッチ表示を同一のステップ番号（インデックス）で一貫して同期できる構造となった。

6.1.2 変数値スナップショットの継承と更新

変数更新ログが「変更された変数のみ」を出力する設計であっても、最新変数表の保持とコピーにより、各ステップにおける変数値スナップショットを再構成できる。これにより、状態ごとの変数値を一貫して参照可能となり、ウォッチ機能として提示できた点が有効性の要因である。

6.1.3 同期モードにおける統合表示

同期モードでは、複数デバイスのログに時刻情報を付与し、統合イベント列を構築することで、同一時刻における状態・変数値の比較を可能とした。この「共通の時間軸に基づく比較」の導入により、分散システムの挙動把握が容易になった点が有効性の要因である。

6.2 システムの限界・制約

6.2.1 ログ設計への依存

本システムは、状態遷移ログ (STATE:) および変数更新ログ ([VAR:]) を前提として解析を行う。したがって、ログ形式が異なる場合には解析器の修正が必要であり、ログ設計への依存が制約となる。

6.2.2 同期タイムラインに関する制約

同期モードの統合は、ログ受信時刻に基づくタイムスタンプ付与により実現しているため、厳密な同時刻性を保証するものではない。この点は、同期表示の解釈における制約となる。

6.3 今後の課題

6.3.1 ログ形式・解析対象の拡張

今後は、ログ形式のバリエーション（例：複数種のイベント、構造化ログ）への対応や、解析対象の拡張（例：エラー原因推定を支援する追加情報）を検討する。

6.3.2 同期精度および比較支援の拡張

同期タイムラインにおける時間整合性の改善や、比較を支援する提示方法（差分強調やイベント関連付け）の拡張が課題である。

6.3.3 状態遷移図表現への対応

状態遷移図の表現力（例：複合状態や履歴状態）に対する対応範囲を整理し、必要に応じて描画・ハイライト手法を拡張することが課題である。

第 7 章

まとめ

7.1 本研究のまとめ

本研究では、無線センサネットワークの実機検証におけるデバッグ作業を支援するため、ブラウザ上で動作するデバッグ支援システムを設計・実装した。本システムは、状態遷移ログおよび変数更新ログを解析し、状態遷移の可視化、変数値追跡、ステップ実行を提供する。さらに、同期モードにより複数ノードのログを統合し、同一時間軸上での比較を可能とした。

7.2 得られた知見と今後の展望

評価を通じて、単一ノードの逐次デバッグおよび複数ノードの同期比較の双方において、状態遷移と変数値を統合的に可視化できることを確認した。今後は、ログ形式・解析対象の拡張、同期精度の改善、および状態遷移図表現への対応範囲の拡張を進める。

参考文献

- [1] IoT Analytics: "State of IoT 2024: Number of Connected IoT Devices Grows 16% to 16.6 Billion", 2024.
- [2] Grand View Research: "Industrial Wireless Sensor Network Market Size, Share & Trends Analysis Report, 2024–2030", 2024.
- [3] 旭 健汰, アサノ デービッド, 不破 泰: "無線モデムを用いたセンサネットワーク MAC プロトコル検証システムの開発", 信学技報, NS2023-147, pp.120–125, Dec. 2023.
- [4] 小林 遼, アサノ デービッド, 不破 泰: "センサーネットワーク検証システムにおける遷移図を用いた MAC プロトコル設計環境の開発", 信学技報, NS2023-148, pp.126–131, Dec. 2023.
- [5] 小林 侑生, アサノ デービッド, 不破 泰:
- [6] PlatformIO Labs, "What is PlatformIO?", <https://docs.platformio.org/en/latest/what-is-platformio.html>, 参照 Oct. 15, 2025.
- [7] Mermaid.js, "Overview," <https://mermaid.js.org/intro/>, 参照 Oct. 15, 2025.
- [8] Mermaid.js, "State Diagram Syntax (stateDiagram-v2)," <https://mermaid.js.org/syntax/stateDiagram.html>, 参照 Oct. 15, 2025.
- [9] Eclipse Foundation, "MQTT: Message Queuing Telemetry Transport," <https://mqtt.org/>, 参照 Oct. 15, 2025.
- [10] 園田 継一郎, 不破 泰, アサノ デービッド, "無線センサーネットワークの端末・中継機における送信タイミング決定時間短縮方法の検討," 信学技報, NS2022-138, Dec. 2022.